

01 · SQL Foundations

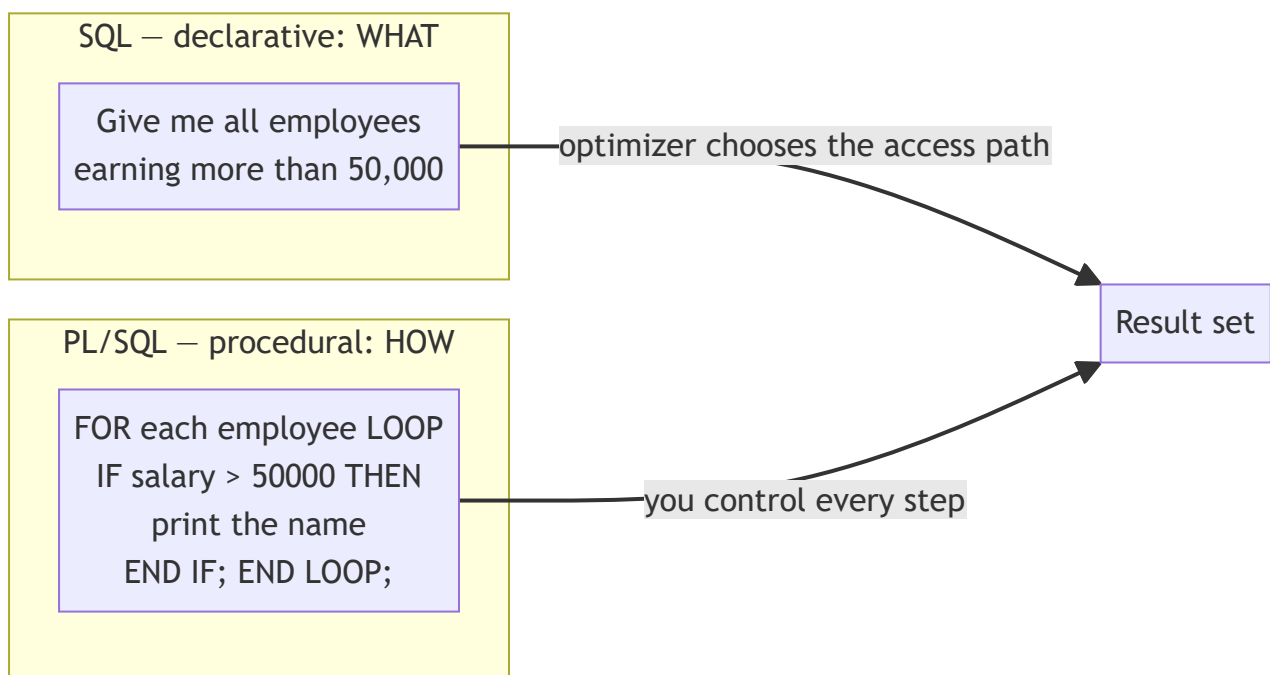
Why this matters in interviews. These are the warm-up questions: *What are DDL, DML, DCL, TCL? CHAR vs VARCHAR2? What is NULL? Why does = NULL return nothing?* They are easy marks when your answer is crisp — and easy to lose when it is vague.

In this chapter: what SQL is · talking to Oracle · the five command families · data types (classic and Oracle) · NULL and three-valued logic · operators · comments, quoting and case rules.

1.1 What SQL is

SQL (Structured Query Language) is the standard language for talking to a relational database. The ISO/ANSI standard defines the core; every vendor (Oracle, MySQL, PostgreSQL, SQL Server) adds its own dialect on top.

SQL is **declarative**: you describe *what* result you want and the database's optimizer decides *how* to get it. PL/SQL (Chapters 12–15) is Oracle's **procedural** extension — there you say *how*, step by step.



Vocabulary you must use correctly

Relational term	Everyday / SQL term	Meaning
Relation	Table	A set of rows with the same columns
Tuple	Row / record	One entry — e.g. one employee
Attribute	Column / field	One property — e.g. <code>salary</code>
Domain	Data type	The set of allowed values for a column
Degree	–	Number of columns
Cardinality	–	Number of rows

Relational term	Everyday / SQL term	Meaning
Schema	Schema	Named collection of objects (tables, views, procedures ...)

Oracle vs others. In Oracle a **schema is the set of objects owned by a user** — creating a user creates a schema of the same name. MySQL uses the word *database* for roughly the same thing, which is why `SHOW DATABASES` exists there and not in Oracle.

1.2 Talking to Oracle: tools and first commands

You will mostly use **Oracle SQL Developer** (GUI) or **SQL*Plus** (command line).

Action in SQL Developer	Shortcut	What you get
Run the one statement under the cursor	Ctrl + Enter	Result grid
Run the whole script	F5	<i>Script Output</i> pane (needed for PL/SQL blocks and <code>SET SERVEROUTPUT ON</code>)
Run a PL/SQL block	end it with a line containing only <code>/</code>	"PL/SQL procedure successfully completed."

Three rules that trip up beginners:

1. `SET SERVEROUTPUT ON;` must be run once per session, otherwise `DBMS_OUTPUT.PUT_LINE` prints nothing.
2. A PL/SQL block ends with `END;` **and then a `/`** on its own line. The `/` is a *client* signal to "send the block now", not part of the language.
3. Commands such as `SHOW`, `DESC`, `SET`, `EXEC`, `VARIABLE`, `PRINT` are **SQL*Plus client commands**, not SQL. They need no semicolon and other databases don't have them.

That last point is exactly why this fails in Oracle:

```
show databases;
```

```
SP2-0158: unknown SHOW option "databases"
```

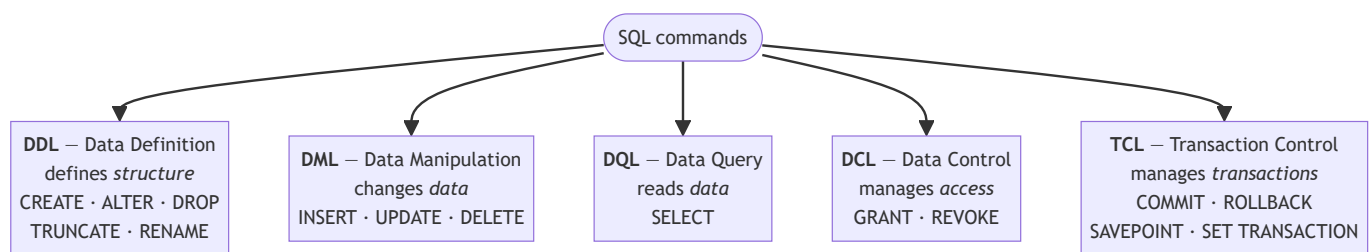
`SHOW DATABASES` is a **MySQL** statement. The Oracle equivalents:

You want to ...	MySQL	Oracle
List databases / schemas	<code>SHOW DATABASES;</code>	<code>SELECT username FROM all_users;</code>
List your tables	<code>SHOW TABLES;</code>	<code>SELECT table_name FROM user_tables;</code>
See a table's columns	<code>DESCRIBE emp;</code>	<code>DESC emp;</code>
Who am I?	<code>SELECT USER();</code>	<code>SHOW USER;</code> or <code>SELECT USER FROM dual;</code>
Switch database	<code>USE shop;</code>	<code>ALTER SESSION SET CURRENT_SCHEMA = shop;</code>
Today's date	<code>SELECT NOW();</code>	<code>SELECT SYSDATE FROM dual;</code>

DUAL is a one-row, one-column dummy table Oracle provides so you can `SELECT` an expression without a real table: `SELECT 2 + 3 FROM dual;` . MySQL, PostgreSQL and SQL Server let you omit `FROM` entirely.

Connection tip (ORA-12505). "Cannot connect to database. SID xe is not registered with the listener" means the listener does not know the SID you typed. On 12c+ multitenant databases you normally connect to a pluggable database by **Service name** (choose the *Service name* option in the connection dialog, e.g. XEPDB1) rather than a SID. Users you create in the container root must be named with the **C##** prefix (e.g. C##PROJECT_USER).

1.3 The five families of SQL commands



Type	Commands	Purpose	Auto-commit?	Can be rolled back?
DDL	CREATE , ALTER , DROP , TRUNCATE , RENAME	Define / change the <i>structure</i> of objects (tables, views, ...)	Yes (Oracle, MySQL)	No
DML	INSERT , UPDATE , DELETE (and SELECT , see below)	Work with the <i>data inside</i> tables	No — needs COMMIT	Yes , until committed
DQL	SELECT	Retrieve data without modifying it	—	—
DCL	GRANT , REVOKE	Give / take back access rights; used by DBAs to secure the database	Yes	No
TCL	COMMIT , ROLLBACK , SAVEPOINT , SET TRANSACTION	Manage transactions (mostly of DML) — protects data integrity in batch processing	—	—

SELECT belongs to two lists. Many textbooks class it under DML; others split it out as **DQL** because it never modifies data. Either answer is accepted — say which convention you are using.

The DDL auto-commit trap

In Oracle every DDL statement **silently commits** the current transaction *before and after* it runs. That is why DDL cannot be rolled back — and why it also makes earlier, uncommitted DML permanent:

```
INSERT INTO emp ...      -- pending, not permanent
UPDATE emp SET ...      -- pending, not permanent
CREATE TABLE audit_log ... -- DDL: COMMITS the two statements above first!
ROLLBACK;               -- too late - there is nothing left to undo
```

Oracle vs others. PostgreSQL and SQL Server have *transactional DDL*: a `CREATE`, `ALTER` or even `TRUNCATE` inside a transaction **can** be rolled back. "DDL is always auto-committed" is true for Oracle and MySQL only.

1.4 Data types

A **data type** fixes what a column can store, how much space it takes and which operations are legal.

1.4.1 The classic table (MySQL-style, from the course video)

Datatype	Description	Usage
CHAR	string (0–255), fixed length	CHAR(50)
VARCHAR	string (0–255 in the video), stores up to the given length	VARCHAR(50)
BLOB	binary large object (up to 65,535 bytes)	BLOB(1000)
INT	integer –2,147,483,648 to 2,147,483,647	INT
TINYINT	integer –128 to 127	TINYINT
BIGINT	integer –9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	BIGINT
BIT	x-bit values, x from 1 to 64	BIT(2)
FLOAT	approximate decimal, single precision (up to 23 bits)	FLOAT
DOUBLE	approximate decimal, double precision (24–53 bits)	DOUBLE
BOOLEAN	0 or 1	BOOLEAN
DATE	YYYY-MM-DD, from 1000-01-01 to 9999-12-31	DATE
YEAR	4-digit year, 1901 to 2155	YEAR

Precision note. The video says `FLOAT` has "23 digits" and `DOUBLE` "24 to 53 digits". Those numbers are **bits of precision**, not decimal digits: a `FLOAT` holds about 7 significant decimal digits, a `DOUBLE` about 15–16. Modern MySQL also allows `VARCHAR` up to 65,535 bytes — the 255 limit in the video is the `CHAR` limit.

1.4.2 Signed vs unsigned integers

An integer type has a fixed number of bits. **Signed** uses one bit for the sign; **unsigned** uses all bits for magnitude, so its maximum is roughly double but it cannot hold negatives.

TINYINT (signed) –128 ————— 0 ————— 127
TINYINT UNSIGNED 0 ————— 255
└ 256 distinct values in both cases (8 bits) ┘

Type	Signed range	UNSIGNED range	Bytes
TINYINT	–128 ... 127	0 ... 255	1
SMALLINT	–32,768 ... 32,767	0 ... 65,535	2

Type	Signed range	UNSIGNED range	Bytes
INT	-2,147,483,648 ... 2,147,483,647	0 ... 4,294,967,295	4
BIGINT	-9.22×10^{18} ... 9.22×10^{18}	0 ... 1.84×10^{19}	8

Oracle vs others. UNSIGNED exists in MySQL only. Oracle has no fixed-width integer types — every integer is the flexible NUMBER (next section), and you use a CHECK (col >= 0) constraint to forbid negatives.

1.4.3 Oracle's data types

Concept	MySQL / standard	Oracle	Notes
Fixed-length text	CHAR(n)	CHAR(n)	Up to 2,000 bytes; blank-padded to length n
Variable-length text	VARCHAR(n)	VARCHAR2(n)	Up to 4,000 bytes in a column (32,767 with MAX_STRING_SIZE=EXTENDED)
Unicode text	NCHAR, NVARCHAR	NCHAR, NVARCHAR2	N = <i>national character set</i> , stores Unicode
Long text	TEXT	CLOB / NCLOB	8 – 128 TB
Binary data	BLOB	BLOB	Stored inside the database, 8 – 128 TB
Binary file outside DB	–	BFILE	Read-only pointer to an OS file, ≤ 4 GB
Integers	TINYINT, INT, BIGINT	NUMBER(p), INTEGER (= NUMBER(38))	No UNSIGNED
Exact decimals	DECIMAL(p, s)	NUMBER(p, s)	DECIMAL, NUMERIC, DEC are aliases
Approximate numbers	FLOAT, DOUBLE	BINARY_FLOAT, BINARY_DOUBLE, FLOAT	FLOAT(p) is a NUMBER with binary precision p
Boolean	BOOLEAN (= TINYINT(1))	<i>none in SQL before 23ai</i>	Use NUMBER(1) or CHAR(1) with a CHECK. PL/SQL <i>does</i> have BOOLEAN
Date only	DATE	DATE ... but it also stores time	see below
Date + time	DATETIME	DATE or TIMESTAMP	
Timestamp	TIMESTAMP	TIMESTAMP [WITH [LOCAL] TIME ZONE]	
Year	YEAR	–	EXTRACT(YEAR FROM d) or NUMBER(4)
Auto number	AUTO_INCREMENT	GENERATED ... AS IDENTITY / SEQUENCE	Chapter 02

VARCHAR VS VARCHAR2

VARCHAR exists in Oracle only for standards compatibility and is today a synonym for VARCHAR2. Oracle has **reserved the right to change VARCHAR's behaviour**, so always write VARCHAR2. VARCHAR2 stores variable-length data of 1–4,000 bytes, keeps trailing spaces you insert, and uses only as much space as the value needs.

CHAR vs VARCHAR2

	CHAR(5)	VARCHAR2(5)
Store 'AB'	stored as 'AB...' (padded with 3 spaces)	stored as 'AB'
LENGTH	5	2
Space used	always 5 bytes	2 bytes (+ length byte)
Best for	truly fixed codes ('Y'/'N' , 'IN' , 'M'/'F')	almost everything else

NUMBER(p, s) — precision and scale

p = total significant digits, **s** = digits to the right of the decimal point (may be **negative** to round to tens, hundreds ...). You can store **p - s** digits to the left of the point.

Column type	Value inserted	Stored	Why
NUMBER(7, 2)	12345.678	12345.68	rounded to 2 decimals
NUMBER(7, 2)	123456.7	error ORA-01438	6 integer digits, only $7 - 2 = 5$ allowed
NUMBER(3)	12.7	13	scale defaults to 0 → rounded
NUMBER(5, -2)	12345	12300	negative scale rounds to hundreds
NUMBER	anything	as given	up to 38 digits of precision

Oracle date and time types

Type	Stores	Precision
DATE	century, year, month, day and hour, minute, second	seconds
TIMESTAMP	a DATE plus fractional seconds	default 6 digits, up to 9
TIMESTAMP WITH TIME ZONE	timestamp and the time-zone offset	as above
TIMESTAMP WITH LOCAL TIME ZONE	timestamp normalised to the database time zone, shown in the session's zone	as above

Valid values for each datetime field (Oracle documentation):

Field	Valid datetime values	Valid <i>interval</i> values
YEAR	-4712 to 9999 (no year 0)	any non-zero integer
MONTH	01 – 12	0 – 11
DAY	01 – 31 (limited by month and year)	any non-zero integer
HOURL	00 – 23	0 – 23
MINUTE	00 – 59	0 – 59
SECOND	00 – 59.9(n), <i>n</i> = fractional precision	0 – 59.9(n)

Date types in MySQL and SQL Server (for comparison)

MySQL	Format	Notes
DATE	YYYY-MM-DD	date only, e.g. 2025-07-26
DATETIME	YYYY-MM-DD HH:MI:SS	date and time, e.g. 2025-07-26 16:30:00
TIMESTAMP	YYYY-MM-DD HH:MI:SS	like DATETIME but stored as UTC ; used for created/updated stamps
YEAR	YYYY or YY	e.g. 2025

SQL Server	Format	Notes
DATE	YYYY-MM-DD	date only
DATETIME	YYYY-MM-DD HH:MI:SS	date and time
SMALLDATETIME	YYYY-MM-DD HH:MI:SS	lower precision and smaller storage than DATETIME
TIMESTAMP	<i>a unique number</i>	⚠ not a date at all — an auto-updated binary row-version counter. Use ROWVERSION

Trap. `TIMESTAMP` means *three different things*: a real date-time in MySQL and Oracle, and a row-version counter in SQL Server. If an interviewer says "TIMESTAMP", ask which database.

The MySQL idiom for an auto-maintained "created / last modified" pair — no Oracle equivalent exists as a column property, so Oracle uses a `DEFAULT SYSTIMESTAMP` plus a `BEFORE UPDATE` trigger (Chapter 15):

```
-- MySQL
CREATE TABLE Users (
  UserID    INT PRIMARY KEY,
  Username  VARCHAR(100),
  CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UpdatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

`CreatedAt` is set once on insert; `UpdatedAt` is refreshed by **every** `UPDATE` of the row.

1.5 NULL and three-valued logic

`NULL` means "**unknown or missing**". It is *not* zero, *not* an empty string and *not* a blank. Because it is unknown, almost every comparison with it is unknown too.

Three-valued logic

A condition in SQL can be **TRUE**, **FALSE** or **UNKNOWN**. A row is returned only when the `WHERE` condition is **TRUE** — UNKNOWN rows are discarded silently.

a	b	a AND b	a OR b
TRUE	TRUE	TRUE	TRUE
TRUE	FALSE	FALSE	TRUE
TRUE	UNKNOWN	UNKNOWN	TRUE
FALSE	FALSE	FALSE	FALSE
FALSE	UNKNOWN	FALSE	UNKNOWN
UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN

a	NOT a
TRUE	FALSE
FALSE	TRUE
UNKNOWN	UNKNOWN

Memory aid: `FALSE AND anything = FALSE` and `TRUE OR anything = TRUE`. In every other combination an UNKNOWN poisons the result.

Testing for NULL

`=`, `<`, `<>` cannot test for NULL, because `NULL = NULL` is itself UNKNOWN — a NULL cannot be equated with anything, **including itself**. Use `IS NULL` / `IS NOT NULL`.

Sample table `t`:

t

id	val
1	10
2	NULL
3	30
4	NULL

```
SELECT id, val FROM t WHERE val = NULL;      -- always empty
```

Result

id	val
(no rows)	

```
SELECT id, val FROM t WHERE val IS NULL;
```

Result

id	val
2	NULL
4	NULL

NULL in expressions and aggregates

```
SELECT id, val, val + 1 AS val_plus_1, NVL(val, 0) AS nvl_val FROM t;
```

Result

id	val	val_plus_1	nvl_val
1	10	11	10
2	NULL	NULL	0
3	30	31	30
4	NULL	NULL	0

Arithmetic with NULL gives NULL; `NVL(x, y)` substitutes `y` when `x` is NULL. Aggregate functions **ignore NULLs**, except `COUNT(*)`:

```
SELECT COUNT(*) AS count_star, COUNT(val) AS count_val,
       SUM(val) AS sum_val, AVG(val) AS avg_val
FROM t;
```

Result

count_star	count_val	sum_val	avg_val
4	2	40	20

AVG is $40 / 2 = 20$, **not** $40 / 4$ — the two NULL rows are not counted.

Where NULLs surprise you

Situation	Behaviour
<code>WHERE col <> 5</code>	rows where <code>col</code> is NULL are excluded (UNKNOWN)
<code>col1 + col2</code> where either is NULL	NULL
<code>ORDER BY col</code> (Oracle)	NULLs sort as the <i>largest</i> value: last for <code>ASC</code> , first for <code>DESC</code> . Override with <code>NULLS FIRST</code> / <code>NULLS LAST</code>
<code>GROUP BY col</code>	all NULLs are grouped together as one group
<code>DISTINCT</code>	treats all NULLs as <i>equal</i> (one NULL survives)
<code>UNIQUE column</code>	Oracle allows several NULLs in a single-column unique constraint
<code>x NOT IN (1, 2, NULL)</code>	never TRUE — returns no rows (see Chapter 08)

Oracle vs others — the empty string. Oracle treats the empty string `''` as **NULL** for `VARCHAR2` / `CHAR`. So `WHERE name = ''` never matches anything and `LENGTH('')` is NULL, not 0. MySQL, PostgreSQL and SQL Server treat `''` as a real, distinct value. Likewise in Oracle `'abc' || NULL` is `'abc'`, while in most other databases it is NULL.

1.6 Operators

Arithmetic

Operator	Meaning	Example	Result
+	Add	10 + 5	15
-	Subtract	10 - 5	5
*	Multiply	10 * 5	50
/	Divide	10 / 4	2.5 (Oracle) — integer division in some databases
%	Modulo (remainder)	10 % 3	1 — Oracle uses MOD(10, 3)

Comparison

Operator	Meaning
=	Equal to
> / <	Greater / less than
>= / <=	Greater / less than or equal
<>	Not equal (!= and ^= also work in Oracle)
BETWEEN a AND b	Within a range, inclusive of both ends
IN (...)	Equal to any value in a list
LIKE	Matches a pattern (% any run of characters, _ exactly one)
IS NULL	Is NULL

Logical

Operator	TRUE when
ALL	all values returned by the subquery satisfy the condition
ANY / SOME	at least one subquery value satisfies the condition (SOME = ANY)
AND	every condition is TRUE
OR	at least one condition is TRUE
NOT	the condition is not TRUE
BETWEEN	the operand lies within the range
EXISTS	the subquery returns at least one row
IN	the operand equals one of a list of expressions
LIKE	the operand matches a pattern

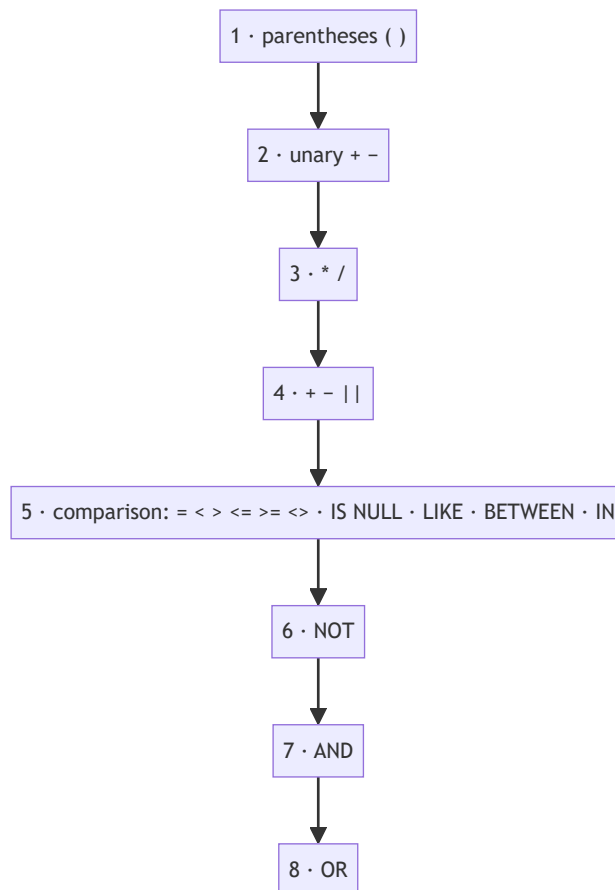
Bitwise and compound operators — mostly a SQL Server / MySQL feature

Kind	Operators	Oracle
Bitwise	<code>&</code> AND · <code>\ </code> OR · <code>^</code> XOR	<code>BITAND(a, b)</code> only; there is no <code>&</code> / <code>\ </code> operator (and <code>&</code> is the <i>substitution-variable</i> prefix in SQL Developer!)
Compound assignment	<code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code> <code>&=</code> <code>^-=</code> <code>\ *=</code>	not supported — write <code>SET x = x + 1</code>

String concatenation

Database	Operator
Oracle, PostgreSQL	<code>\ \ </code> (and <code>CONCAT(a, b)</code> — two arguments only in Oracle)
MySQL	<code>CONCAT(a, b, c, ...)</code>
SQL Server	<code>+</code> (and <code>CONCAT</code>)

Precedence (highest → lowest)



Because `AND` binds tighter than `OR` , always **parenthesise mixed conditions** (worked example in Chapter 04).

1.7 Comments, quoting and case rules

Comments

```
-- single-line comment: everything to the end of the line is ignored
SELECT * FROM emp;           -- can follow a statement

/* multi-line comment:
   everything between the markers is ignored */
SELECT * FROM emp;
```

`/+ ... */` is not an ordinary comment in Oracle — it is an **optimizer hint**. Comments cannot be nested.

Quotes: single vs double

Quotes	Used for	Example
'single'	string and date literals	WHERE country = 'India'
"double"	identifiers (column/table/alias names) that contain spaces or must keep their case	SELECT salary AS "Monthly Pay"

A single quote *inside* a string is written twice: `'it's'`. Numbers are never quoted (WHERE id = 1).

```
SELECT 'It's Oracle' FROM dual;           -- It's Oracle
SELECT q'[It's Oracle]' FROM dual;       -- alternative quoting: no doubling needed
```

Case sensitivity

Thing	Case-sensitive?
Keywords (select , SELECT)	No
Unquoted identifiers (emp , EMP)	No — Oracle stores them in UPPERCASE
Quoted identifiers ("Emp")	Yes — must always be quoted afterwards
String data ('Alice' vs 'alice')	Yes in Oracle — WHERE name = 'alice' does not match Alice (MySQL's default collation is case-insensitive)

Identifier rules (Oracle): start with a letter; then letters, digits, `_`, `$`, `#`; up to 30 bytes (128 from 12.2); cannot be a reserved word.

Interview corner

Q. What are DDL, DML, DCL, TCL and DQL? Give commands. DDL defines structure (CREATE ALTER DROP TRUNCATE RENAME); DML changes data (INSERT UPDATE DELETE); DQL reads it (SELECT); DCL controls access (GRANT REVOKE); TCL controls transactions (COMMIT ROLLBACK SAVEPOINT SET TRANSACTION). DDL auto-commits; DML can be rolled back until committed.

Q. Why can't you roll back a CREATE TABLE in Oracle? Every DDL statement issues an implicit COMMIT before and after it, so there is no open transaction left to roll back.

Q. CHAR vs VARCHAR2 ? CHAR(n) is fixed length and blank-pads to n ; VARCHAR2(n) is variable length and stores only what you give it. Use CHAR only for genuinely fixed-width codes.

Q. VARCHAR vs VARCHAR2 in Oracle? Today they behave the same, but `VARCHAR` is reserved for future standard-conforming behaviour — use `VARCHAR2`.

Q. Why does `WHERE col = NULL` return nothing? Comparing with `NULL` yields `UNKNOWN`, and `WHERE` keeps only `TRUE` rows. Use `IS NULL`.

Q. What is the difference between `NULL`, `0` and an empty string? `NULL` is "unknown/missing"; `0` is a number; `''` is a zero-length string — except in Oracle, where `''` is `NULL`.

Q. Does `COUNT(col)` count `NULL`s? No. `COUNT(*)` counts rows; `COUNT(col)` counts rows where `col` is not `NULL`.

Q. What is `DUAL`? A single-row dummy table used to `SELECT` expressions or functions that don't come from a real table.

Quick recap

- SQL is **declarative** (what); PL/SQL is **procedural** (how).
- **DDL** structure · **DML** data · **DQL** `SELECT` · **DCL** access · **TCL** transactions. DDL auto-commits.
- Oracle text type is `VARCHAR2`; numbers are `NUMBER(p,s)`; `DATE` **includes time**; no SQL `BOOLEAN` before 23ai.
- `NULL` $\neq$ `0` $\neq$ `''` (except Oracle, where `''` **is** `NULL`). Test with `IS NULL`; a `WHERE` keeps only `TRUE` rows.
- Aggregates ignore `NULL`s (except `COUNT(*)`); `NOT IN` with a `NULL` in the list returns nothing.
- Single quotes for values, double quotes for identifiers; Oracle string comparisons are case-sensitive.